

Plan de Branching - Tienda Retail Lumina

Proyecto: FullStack-III

1. Estrategia de Branching Seleccionada

Se utilizo una estrategia basada en **GitHub Flow simplificado**, adaptada al contexto del proyecto academico con un equipo reducido.

2. Estructura de Ramas

2.1 Rama Principal

Rama	Proposito
main	Rama principal y de produccion. Contiene el codigo estable y funcional del proyecto.

2.2 Flujo de Trabajo

```
main (produccion/estable)
├── feature/nombre-feature  ← Desarrollo de nueva funcionalidad
│   └── merge → main        ← Una vez probado se integra
├── fix/nombre-fix          ← Correccion de errores
│   └── merge → main
└── docs/documentacion      ← Documentacion del proyecto
    └── merge → main
```

3. Convenciones de Nombres de Ramas

Prefijo	Uso	Ejemplo
feature/	Nueva funcionalidad	feature/stock-alerts
fix/	Correccion de bugs	fix/duplicate-alerts
docs/	Documentacion	docs/readme-update
refactor/	Refactorizacion de codigo	refactor/service-layer

4. Flujo de Integracion

4.1 Desarrollo de una Feature

1. Crear rama desde `main`:

```
git checkout -b feature/nombre-feature main
```

2. Desarrollar la funcionalidad con commits descriptivos:

```
git commit -m "feat: implementar gestion de alertas de stock"
```

3. Probar localmente con Docker Compose:

```
docker-compose up -d --build
```

4. Integrar a `main`:

```
git checkout main
git merge feature/nombre-feature
git push origin main
```

4.2 Correccion de Bugs

1. Crear rama de fix desde `main`:

```
git checkout -b fix/nombre-bug main
```

2. Corregir el bug y verificar.

3. Integrar a `main` con commit descriptivo.

5. Convenciones de Commits

Se sigue el estandar **Conventional Commits**:

Prefijo	Descripcion	Ejemplo
feat:	Nueva funcionalidad	feat: agregar descuento de stock en pago
fix:	Correccion de bug	fix: alertas duplicadas en bodega
docs:	Documentacion	docs: actualizar README con instrucciones
refactor:	Refactorizacion	refactor: separar logica de alertas

Prefijo	Descripcion	Ejemplo
style:	Cambios de estilo/UI	style: alinear iconos en registro
chore:	Tareas de mantenimiento	chore: actualizar dependencias

6. Justificacion de la Estrategia

¿Por que GitHub Flow?

1. **Simplicidad:** Al ser un equipo reducido en contexto academico, una estrategia sencilla permite iterar rapidamente sin la complejidad de GitFlow.
2. **Rama principal siempre estable:** `main` siempre contiene codigo funcional y desplegable.
3. **Integracion continua:** Los cambios se integran frecuentemente, evitando conflictos grandes.
4. **Adaptado al proyecto:** Con Docker Compose, cada rebuild es rapido, por lo que no se necesitan ramas de staging/release intermedias.

Alternativas consideradas

Estrategia	Razon de descarte
GitFlow	Demasiado complejo para el tamano del equipo y la duracion del proyecto.
Trunk-Based	Requiere CI/CD automatizado mas robusto del que se dispone.

7. Repositorio

- URL: <https://github.com/YutaOkkotsuUwU/FullStack-III>
- Rama principal: `main`
- Todos los componentes (frontend, BFF, microservicios) se encuentran en un monorepo para facilitar la gestion con Docker Compose.

8. Diagrama Resumen

